

An Elementary Method For Fast Modular Exponentiation With Factored Modulus

Breaking the $\log m$ barrier

Manu Isaacs, Anay Aggarwal

Jesuit High School, Portland, OR, USA

Introduction

- Modular exponentiation is the problem of computing the remainder when a^n is divided by m for an object a and natural numbers n and m for which this is defined. We denote this remainder by $a^n \pmod{m}$.
- This problem became important in practice when the RSA cryptosystem was introduced. It is useful because the inverse problem, the discrete logarithm problem, is computationally hard. This makes for secure systems.
- Since then, it has shown up in many different places in information security, such as error-correcting codes, primality testing, and quantum algorithms (like Shor’s algorithm).
- We discovered and analyzed a fast algorithm for modular exponentiation** when the prime factorization m is known. Our algorithm extends to matrices, linear sequences, and even more types of general algebraic structures, like $\mathbb{Z}[i]$.
- The fastest known algorithms for matrix and integer modular exponentiation (without precomputation) have the same asymptotic complexity as repeated squaring, which takes on the order of $\log m$ steps (where steps are multiplications modulo m).
- The fastest known algorithms for computing terms of linear recurrences in rings are not asymptotically faster than Fiduccia’s algorithm. This is also on the order of $\log m$ steps, as a function of m .

Definitions

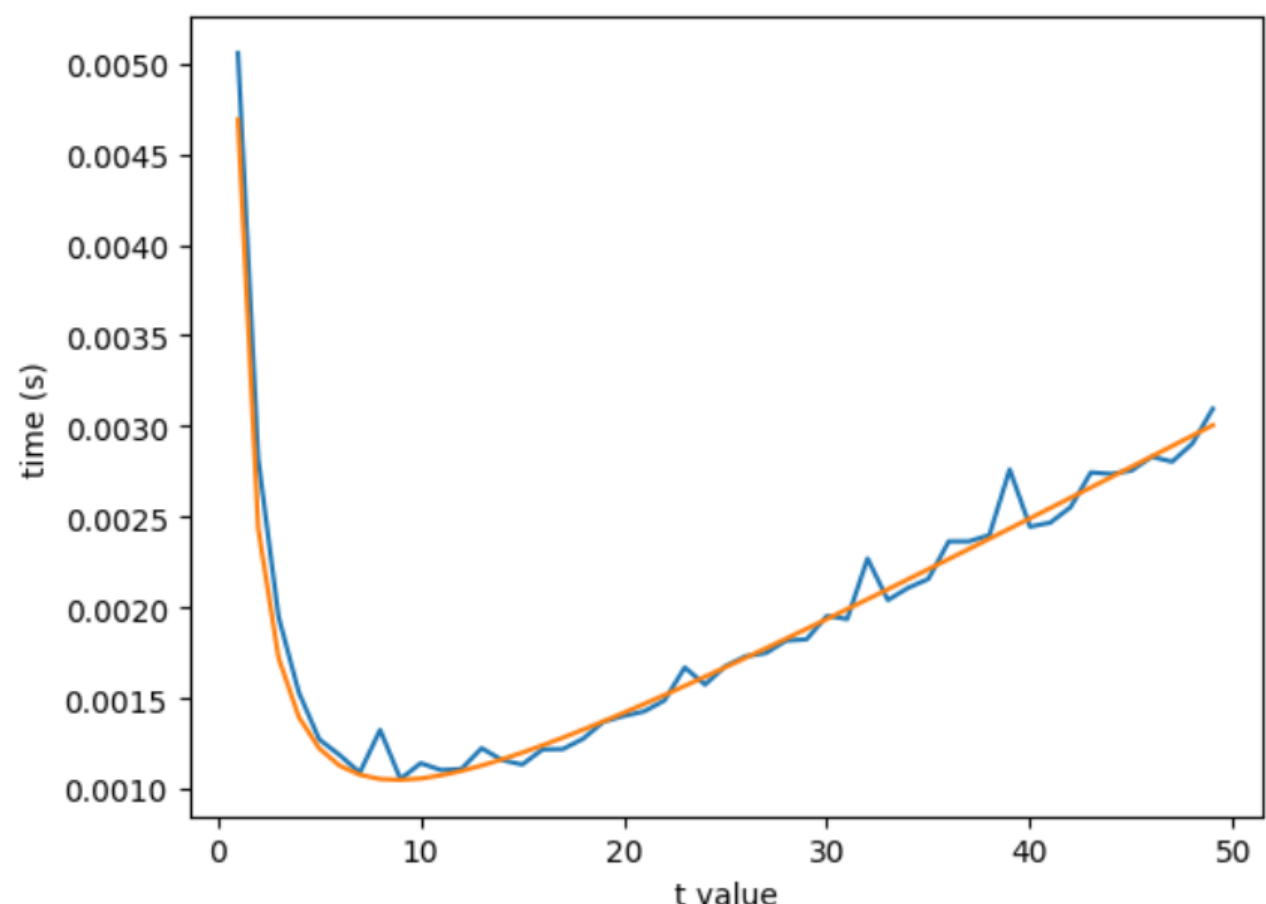
- A function $f(n)$ is $O(g(n))$ if $f(n) \leq Cg(n)$ for a constant C and all sufficiently large n .
- $GL_d(R)$ is the general linear group of $d \times d$ invertible matrices over the ring R .
- An order d linear recurrence is one such that each term is a linear combination of the previous d terms.

Applications

- Our algorithm potentially provides **fast encryption and decryption** for RSA-type cryptosystems like Takagi’s cryptosystem (an RSA-variant with a modulus more amenable to our algorithm).
- Our algorithm **improves new variants** of cryptosystems that use exponent-heavy moduli.
- The matrix and linear sequence versions of our algorithm contribute to **fast encoding** for an actively researched family of error-correcting codes.
- Our algorithm provides for **fast computation** of terms of many families of pseudorandom number generators.

Parameter Optimization

- Our algorithm involves a set of input parameters. While it can be proven that we want these parameters to be small, the choice between small values is vital. The below figure illustrates this, as we iterate over input t parameters in a particular case. Notice the degree 2 curve fit, which we’ve proven mathematically.



- We’ve tried approaches involving Neural Networks as well as gradient descent methods like the Nelder-Mead Algorithm, to various levels of success. See our GitHub for more details.

Main Results

We are the first to break the barrier of the asymptotic $O(\log m)$ complexity. The key idea behind our algorithm is to allow for a kind of Hensel lifting using a clever decomposition of the exponent (which involves a set of input parameters). Along with recursive computation of binomial coefficients, this is quite fast. We present results on its asymptotic complexity.

The setup for all variants of our algorithm is to begin with a positive integer m with known factorization $\prod_{i=1}^k p_i^{e_i}$. We choose positive integers a, n such that $\gcd(a, m) = 1$. Now, we require k input parameters: for each $1 \leq i \leq k$ we must pick some integer $t_i \in [0, e_i]$. Our main algorithm gives the following:

Complexity Of Main Algorithm

We may compute $a^n \pmod{m}$ in

$$O\left(\max(e_i/t_i) + \sum_{i=1}^k t_i \log p_i\right)$$

steps, where steps are multiplications modulo m .

Denoting $\ell = \max(e_i/t_i)$, this algorithm requires $O(\ell)$ memory. This is negligible in general, but we have a variant of our algorithm which is $O(1)$ memory. The cost is an extra term of $\ell \log \ell$.

It turns out that with the same setup, our algorithm generalizes to matrices and general linear recurrences.

Complexity Of Matrix Generalization

Let A be a matrix in $GL_d(\mathbb{Z}/m\mathbb{Z})$. We may compute A^n in

$$O\left(\max(e_i/t_i) + \sum_{j=1}^k (t_j - 1)d^2 \log p_j + \sum_{j=1}^k \sum_{i=0}^{d-1} \log(p_j^d - p_j^i)\right) < O\left(\max(e_i/t_i) + d^2 \sum_{i=1}^k t_i \log p_i\right)$$

modular matrix multiplications.

It follows that we may compute terms of most linear recurrences of order d in the same complexity as given above for matrices:

Complexity Of Linear Recurrence Version

Let $(u_n)_{n=1}^\infty$ be a known linear recurrence of order d such that the lowest coefficient is relatively prime with m . We may compute **any term** of u in the same complexity as the above matrix result.

These results improve upon the state-of-the-art results (such as Fiduccia’s algorithm) for $\mathbb{Z}/m\mathbb{Z}$ in most cases, such as $m \gg d$.

Additionally, **the idea behind our algorithm is extremely versatile and can be used in even more general contexts**. For example, we can perform modular exponentiation in the ring of Gaussian Integers, $\mathbb{Z}[i]$, in the same complexity as our main algorithm.

Comparison to Repeated Squaring

- For general moduli, it can be proven that it is optimal to set all input parameters to be $O(1)$.
- In this case, we can use analytic number theory to compute average-order metrics compared to repeated squaring. It turns out that we **do make asymptotic improvements** by certain metrics.
- However, for particular m , we make significant improvement. When m has large exponents compared to primes, we have **large boosts in asymptotic complexity** compared to repeated squaring.
- We call m *exponent-heavy* if all p_i have the same bit length P and all exponents e_i are $ck \log P$ for a constant c .

Steps For Particular Moduli

If m is exponent-heavy, we can compute $a^n \pmod{m}$ in $O(\sqrt{\log m})$ steps.

We make square-root improvement – incredible! Steps don’t accurately reflect time, however, because one modular multiplication takes a non-negligible amount of bit operations. With unoptimized modular multiplication, we have the following result.

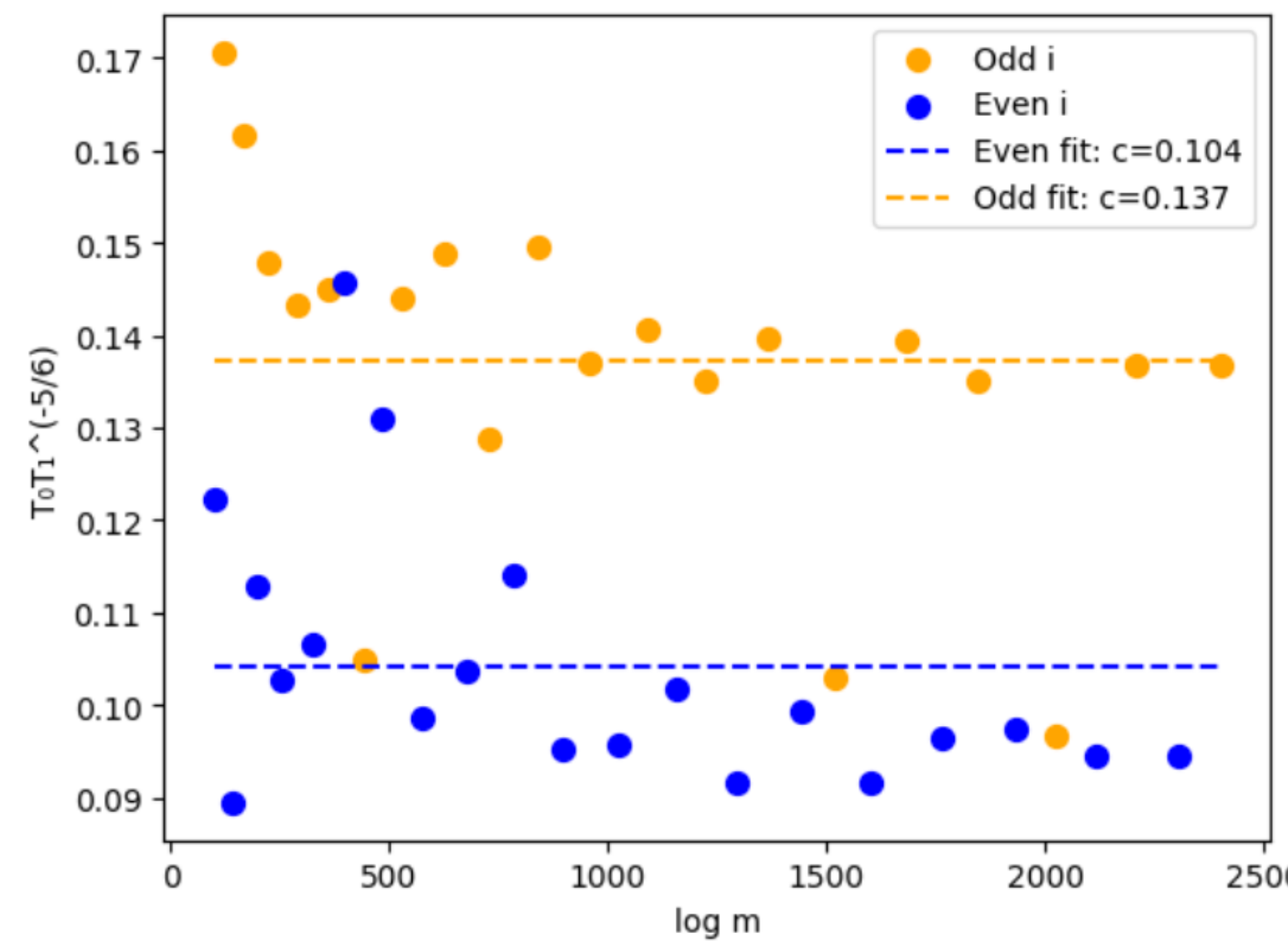
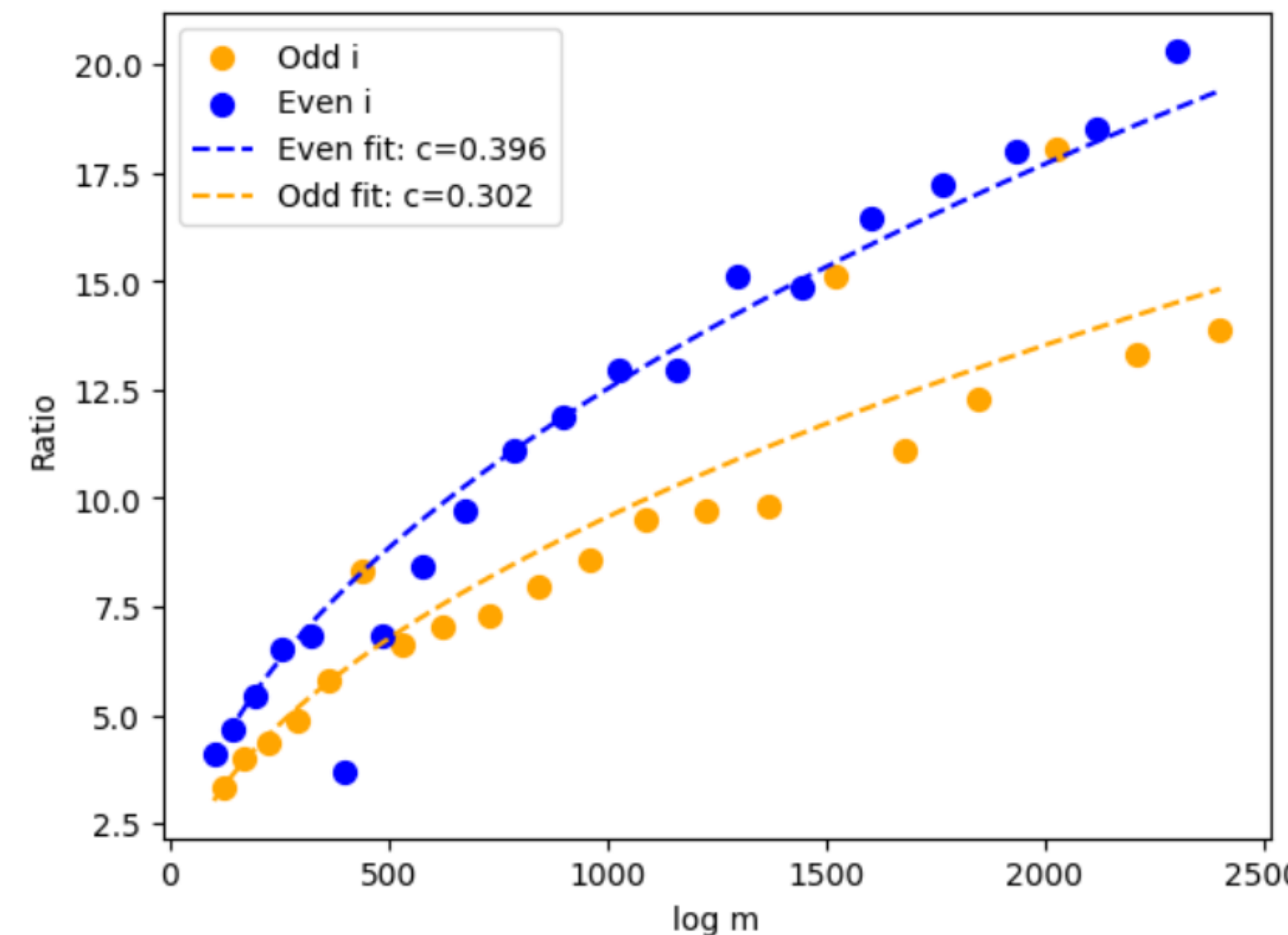
Time For Particular Moduli

Suppose that m is exponent-heavy. If, for some a, n , repeated squaring modulo m takes T time, our algorithm runs on the order of $T^{5/6}$ time.

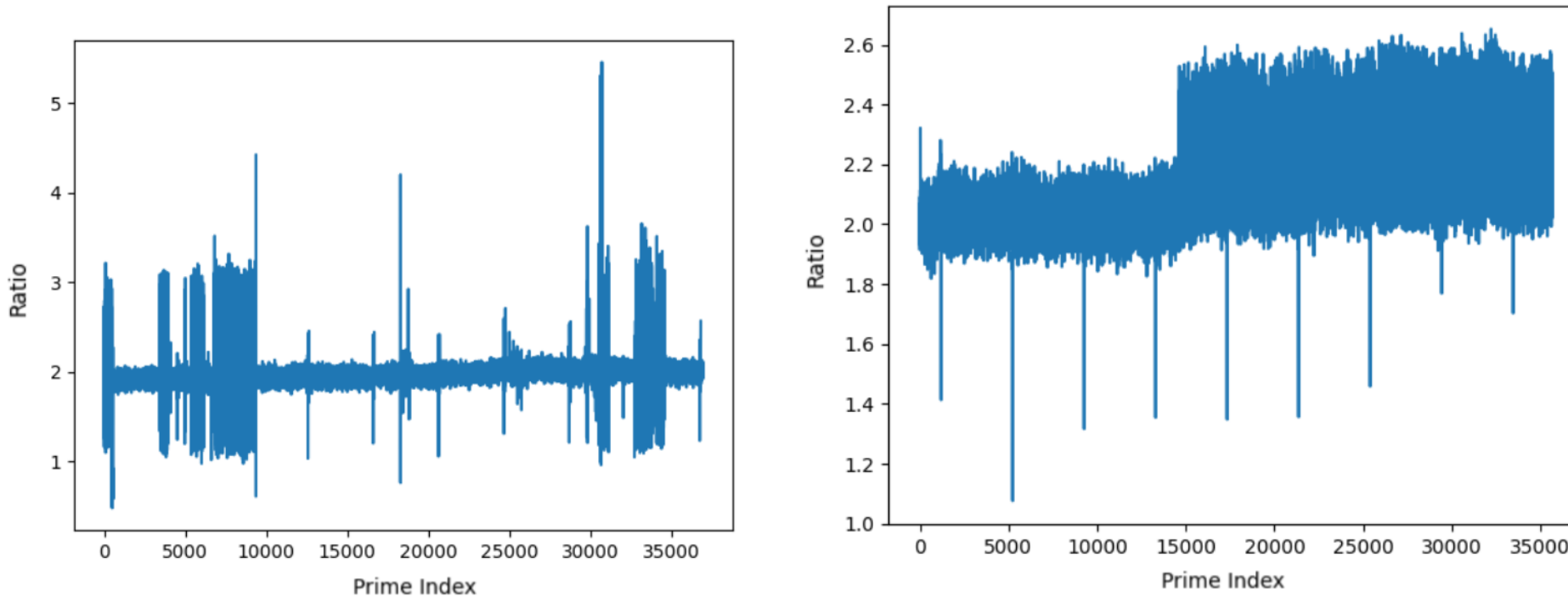
The exponent of $5/6$ can be reduced further with modular multiplication optimizations such as Montgomery Multiplication and the Fast Fourier Transform.

Empirical Analysis

- We provide empirical verification for our theoretical results. Here we present four graphs, more visuals can be found on our GitHub.
- We test against Python’s optimized pow function, computing the ratio of the time it takes.
- To graph $\log m$ compared to the ratio, we set m as the smallest prime greater than 10^i raised to the power of i and iterate over i (we expect this to be “random” to some extent).
- We choose other parameters of the modular exponentiation randomly, and set all $t_j = 1$.
- The first figure verifies our $O(\sqrt{\log m})$ asymptotic, and the second verifies our $O(T^{5/6})$ asymptotic.



- Notice that there is a distinct curve fit for odd and even values of i . We are excited by this phenomenon as we are still unable to explain it mathematically.
- We may also test over general prime power moduli, iterating over all primes in certain intervals. The left figure shows the ratio for small primes, and the right figure shows the ratio for larger primes. Exact details of the setup can be found on our GitHub.



- As seen, the ratio starts off with high variance, but the deviation reduces as we work with larger primes.

Future Work

- We have found and proven an algorithm for fast modular exponentiation when the factorization of the modulus is known. This algorithm extends to general cases and has applications in secure data transmission.
- We plan to work further on the front of optimizing input parameters.
- We hope to devise a better framework for general empirical analysis of our algorithm (beyond testing just prime power moduli).
- We hope to explain some of the phenomena in our figures.
- We plan to explicitly apply our algorithm in practice.